

Practice Material - Semantics

- * 1. For each of the following states $\sigma_1, \sigma_2, \sigma_3$, calculate: (a) the denotation of the arithmetic expression $x + (y * 3)$; (b) the denotation of the Boolean expression $x == y \ \&\& \ x - y \leq 3$; (c) And, for each $i \in \{1, 2, 3\}$, a state σ such that $\langle \text{while } x \leq y \ \&\& \ !(x = y) \text{ do } x \leftarrow x + z, \sigma_i \rangle \rightarrow^* \sigma$ when one exists.
1. $\sigma_1 = [x \mapsto 0, y \mapsto 0]$
 2. $\sigma_2 = [x \mapsto -1, y \mapsto 10]$
 3. $\sigma_3 = [x \mapsto -1, z \mapsto 10]$

Solution

- (a) The arithmetic expression $x + (y * 3)$ evaluates to:
- (a) 0 in the state $[x \mapsto 0, y \mapsto 0]$
 - (b) 29 in the state $[x \mapsto -1, y \mapsto 10]$
 - (c) -1 in the state $[x \mapsto -1, z \mapsto 10]$
- (b) The Boolean expression evaluates $x == y \ \&\& \ x - y \leq 3$ to:
- (a) $\top$ in the state $[x \mapsto 0, y \mapsto 0]$
 - (b) $\perp$ in the state $[x \mapsto -1, y \mapsto 10]$
 - (c) $\perp$ in the state $[x \mapsto -1, z \mapsto 0]$
- (c) The statement while $x \leq y \ \&\& \ !(x = y)$ do $x \leftarrow x + z$
- (a) $[x \mapsto 0, y \mapsto 0, z \mapsto 0]$
 - (b) No such state exists, the program does not terminate.
 - (c) $[x \mapsto 9, y \mapsto 0, z \mapsto 10]$

- ** 2. Suppose $e \in \mathcal{B}$ is an arbitrary Boolean expression and $S_1, S_2, S_3 \in \mathcal{S}$ are arbitrary statements. Prove that $\langle \text{if } e \text{ then } S_1 \text{ else } S_2, \sigma \rangle \rightarrow^* \sigma'$ if, and only if, $\langle \text{if } e \text{ then } \{\text{if } e \text{ then } S_1 \text{ else } S_3\} \text{ else } \{\text{if } !e \text{ then } S_2 \text{ else } S_3\} \rangle \rightarrow^* \sigma'$ for any states $\sigma, \sigma' \in \text{State}$.

Solution

First, let us suppose that $\langle \text{if } e \text{ then } S_1 \text{ else } S_2, \sigma \rangle \rightarrow^* \sigma'$ for two states $\sigma, \sigma' \in \text{State}$. By definition, one of the following must apply:

- Either $\llbracket e \rrbracket_{\mathcal{B}}(\sigma) = \top$ and $\langle S_1, \sigma \rangle \rightarrow^* \sigma'$. In this case, we can construct the following trace:

$$\begin{aligned} & \langle \text{if } e \text{ then } \{ \text{if } e \text{ then } S_1 \text{ else } S_3 \} \text{ else } \{ \text{if } !e \text{ then } S_2 \text{ else } S_3 \} \rangle \\ & \rightarrow \langle \text{if } e \text{ then } S_1 \text{ else } S_3, \sigma \rangle \\ & \rightarrow \langle S_1, \sigma \rangle \\ & \rightarrow^* \sigma' \end{aligned}$$

as required.

- Or, $\llbracket e \rrbracket_{\mathcal{B}}(\sigma) = \perp$ and $\langle S_2, \sigma \rangle \rightarrow^* \sigma'$ in which case we can construct the following trace:

$$\begin{aligned} & \langle \text{if } e \text{ then } \{ \text{if } e \text{ then } S_1 \text{ else } S_3 \} \text{ else } \{ \text{if } !e \text{ then } S_2 \text{ else } S_3 \} \rangle \\ & \rightarrow \langle \text{if } !e \text{ then } S_2 \text{ else } S_3, \sigma \rangle \\ & \rightarrow \langle S_2, \sigma \rangle \\ & \rightarrow^* \sigma' \end{aligned}$$

as $\llbracket !e \rrbracket_{\mathcal{B}}(\sigma) = \neg \llbracket e \rrbracket_{\mathcal{B}}(\sigma) = \top$.

In the converse direction, let us suppose that $\langle \text{if } e \text{ then } \{ \text{if } e \text{ then } S_1 \text{ else } S_3 \} \text{ else } \{ \text{if } !e \text{ then } S_2 \text{ else } S_3 \} \rangle \rightarrow^* \sigma'$. Again by definition, there are two cases to consider:

- Either $\llbracket e \rrbracket_{\mathcal{B}}(\sigma) = \top$ and $\langle \text{if } e \text{ then } S_1 \text{ else } S_3, \sigma \rangle \rightarrow^* \sigma'$. Therefore, $\langle S_1, \sigma \rangle \rightarrow^* \sigma'$ as the other case (where $\llbracket e \rrbracket_{\mathcal{B}}(\sigma) = \perp$) would give us a contradiction. Therefore, we can construct the following derivation:

$$\begin{aligned} & \langle \text{if } e \text{ then } S_1 \text{ else } S_2, \sigma \rangle \\ & \rightarrow \langle S_1, \sigma \rangle \\ & \rightarrow^* \sigma' \end{aligned}$$

as required.

- Or $\llbracket e \rrbracket_{\mathcal{B}}(\sigma) = \perp$ and $\langle \text{if } !e \text{ then } S_2 \text{ else } S_3, \sigma \rangle \rightarrow^* \sigma'$. Again, we have that $\langle S_2, \sigma \rangle \rightarrow^* \sigma'$ as the other case (where $\llbracket !e \rrbracket_{\mathcal{B}}(\sigma) = \perp$) would give us a contradiction. Therefore, we can construct the following derivation:

$$\begin{aligned} & \langle \text{if } e \text{ then } S_1 \text{ else } S_2, \sigma \rangle \\ & \rightarrow \langle S_2, \sigma \rangle \\ & \rightarrow^* \sigma' \end{aligned}$$

as required.

**** 3.** Let $\mathbb{P}$ be the three element set $\{+, -, \pm\}$. We define the function $\text{sign}_{\sigma} : \mathcal{A} \rightarrow \mathbb{P}$ for a given state

$\sigma \in \text{State}$ by recursion as follows:

$$\begin{aligned} \text{sign}_\sigma(x) &= \begin{cases} + & \text{if } \sigma(x) \geq 0 \\ - & \text{otherwise} \end{cases} \\ \text{sign}_\sigma(n) &= \begin{cases} + & \text{if } n \geq 0 \\ - & \text{otherwise} \end{cases} \\ \text{sign}_\sigma(e_1 + e_2) &= \begin{cases} + & \text{if } \text{sign}_\sigma(e_1) = + \text{ and } \text{sign}_\sigma(e_2) = + \\ - & \text{if } \text{sign}_\sigma(e_1) = - \text{ and } \text{sign}_\sigma(e_2) = - \\ \pm & \text{otherwise} \end{cases} \\ \text{sign}_\sigma(e_1 - e_2) &= \begin{cases} + & \text{if } \text{sign}_\sigma(e_1) = + \text{ and } \text{sign}_\sigma(e_2) = - \\ - & \text{if } \text{sign}_\sigma(e_1) = - \text{ and } \text{sign}_\sigma(e_2) = + \\ \pm & \text{otherwise} \end{cases} \\ \text{sign}_\sigma(e_1 * e_2) &= \begin{cases} \pm & \text{if } \text{sign}_\sigma(e_1) = \pm \text{ or } \text{sign}_\sigma(e_2) = \pm \\ + & \text{if } \text{sign}_\sigma(e_1) = \text{sign}_\sigma(e_2) \\ + & \text{if } \text{sign}_\sigma(e_1) \neq \text{sign}_\sigma(e_2) \end{cases} \end{aligned}$$

Prove by structural induction over arithmetic expressions, for any arithmetic $e \in \mathcal{A}$ and state $\sigma \in \text{State}$, that:

- If $\text{sign}_\sigma(e) = +$, then $\llbracket e \rrbracket_{\mathcal{A}}(\sigma) \geq 0$;
- And, if $\text{sign}_\sigma(e) = -$, then $\llbracket e \rrbracket_{\mathcal{A}}(\sigma) < 0$

You should treat this as a single induction proof rather than proving each cases separately.

Solution

Let us proceed by structural induction on arithmetic expressions.

- Consider an expression of the form $n \in \mathbb{Z}$:
 - If $\text{sign}_\sigma(n) = +$, then by definition $n \geq 0$ and thus $\llbracket n \rrbracket_{\mathcal{A}}(\sigma) = n \geq 0$ as required.
 - Likewise, if $\text{sign}_\sigma(n) = -$, then by definition $n < 0$ and thus $\llbracket n \rrbracket_{\mathcal{A}}(\sigma) = n < 0$ as required.
- Consider an expression of the form $x \in \text{Var}$:
 - If $\text{sign}_\sigma(x) = +$, then by definition $\sigma(x) \geq 0$ and thus $\llbracket x \rrbracket_{\mathcal{A}}(\sigma) = \sigma(x) \geq 0$ as required.
 - Likewise, if $\text{sign}_\sigma(x) = -$, then by definition $\sigma(x) < 0$ and thus $\llbracket x \rrbracket_{\mathcal{A}}(\sigma) = \sigma(x) < 0$ as required.
- Consider an expression of the form $e_1 + e_2$:
 - If $\text{sign}_\sigma(e_1 + e_2) = +$, then by definition $\text{sign}_\sigma(e_1) = +$ and $\text{sign}_\sigma(e_2) = +$. Therefore, by induction, $\llbracket e_1 \rrbracket_{\mathcal{A}}(\sigma), \llbracket e_2 \rrbracket_{\mathcal{A}}(\sigma) \geq 0$. It thus follows that $\llbracket e_1 + e_2 \rrbracket_{\mathcal{A}}(\sigma) = \llbracket e_1 \rrbracket_{\mathcal{A}}(\sigma) + \llbracket e_2 \rrbracket_{\mathcal{A}}(\sigma) \geq 0$ as required.
 - If $\text{sign}_\sigma(e_1 + e_2) = -$, then by definition $\text{sign}_\sigma(e_1) = -$ and $\text{sign}_\sigma(e_2) = -$. Therefore, by induction, $\llbracket e_1 \rrbracket_{\mathcal{A}}(\sigma), \llbracket e_2 \rrbracket_{\mathcal{A}}(\sigma) < 0$. It thus follows that $\llbracket e_1 + e_2 \rrbracket_{\mathcal{A}}(\sigma) = \llbracket e_1 \rrbracket_{\mathcal{A}}(\sigma) + \llbracket e_2 \rrbracket_{\mathcal{A}}(\sigma) < 0$ as required.

- Consider an expression of the form $e_1 - e_2$:
 - If $\text{sign}_\sigma(e_1 - e_2) = +$, then by definition $\text{sign}_\sigma(e_1) = +$ and $\text{sign}_\sigma(e_2) = -$. Therefore, by induction, $\llbracket e_1 \rrbracket_{\mathcal{A}}(\sigma) \geq 0$ and $\llbracket e_2 \rrbracket_{\mathcal{A}}(\sigma) < 0$. It thus follows that $\llbracket e_1 - e_2 \rrbracket_{\mathcal{A}}(\sigma) = \llbracket e_1 \rrbracket_{\mathcal{A}}(\sigma) - \llbracket e_2 \rrbracket_{\mathcal{A}}(\sigma) \geq 0$ as required.
 - If $\text{sign}_\sigma(e_1 - e_2) = -$, then by definition $\text{sign}_\sigma(e_1) = -$ and $\text{sign}_\sigma(e_2) = +$. Therefore, by induction, $\llbracket e_1 \rrbracket_{\mathcal{A}}(\sigma) < 0$ and $\llbracket e_2 \rrbracket_{\mathcal{A}}(\sigma) \geq 0$. It thus follows that $\llbracket e_1 - e_2 \rrbracket_{\mathcal{A}}(\sigma) = \llbracket e_1 \rrbracket_{\mathcal{A}}(\sigma) - \llbracket e_2 \rrbracket_{\mathcal{A}}(\sigma) < 0$ as required.
- Consider an expression of the form $e_1 * e_2$:
 - If $\text{sign}_\sigma(e_1 * e_2) = +$, then by definition $\text{sign}_\sigma(e_1) = +$ and $\text{sign}_\sigma(e_2) = +$ or $\text{sign}_\sigma(e_1) = -$ and $\text{sign}_\sigma(e_2) = -$. If both have the sign $+$, then, by induction, $\llbracket e_1 \rrbracket_{\mathcal{A}}(\sigma), \llbracket e_2 \rrbracket_{\mathcal{A}}(\sigma) \geq 0$ and thus it follows that $\llbracket e_1 * e_2 \rrbracket_{\mathcal{A}}(\sigma) = \llbracket e_1 \rrbracket_{\mathcal{A}}(\sigma) \cdot \llbracket e_2 \rrbracket_{\mathcal{A}}(\sigma) \geq 0$ as required. Otherwise, if both have the sign $-$, then, by induction, $\llbracket e_1 \rrbracket_{\mathcal{A}}(\sigma), \llbracket e_2 \rrbracket_{\mathcal{A}}(\sigma) < 0$ and thus it follows that $\llbracket e_1 * e_2 \rrbracket_{\mathcal{A}}(\sigma) = \llbracket e_1 \rrbracket_{\mathcal{A}}(\sigma) \cdot \llbracket e_2 \rrbracket_{\mathcal{A}}(\sigma) < 0$ as required.
 - If $\text{sign}_\sigma(e_1 * e_2) = -$, then by definition $\text{sign}_\sigma(e_1) = -$ and $\text{sign}_\sigma(e_2) = +$ or vice versa. Let us suppose that $\text{sign}_\sigma(e_1) = -$ and $\text{sign}_\sigma(e_2) = +$. By induction, $\llbracket e_1 \rrbracket_{\mathcal{A}}(\sigma) < 0$ and $\llbracket e_2 \rrbracket_{\mathcal{A}}(\sigma) \geq 0$. It thus follows that $\llbracket e_1 * e_2 \rrbracket_{\mathcal{A}}(\sigma) = \llbracket e_1 \rrbracket_{\mathcal{A}}(\sigma) \cdot \llbracket e_2 \rrbracket_{\mathcal{A}}(\sigma) < 0$ as required. The case where $\text{sign}_\sigma(e_1) = +$ and $\text{sign}_\sigma(e_2) = -$ is analogous.

The following question uses the notation $\sigma \sim_x \sigma'$ which indicates, for some variable $x \in \text{Var}$ and for two states $\sigma, \sigma' \in \text{State}$, that $\sigma(y) = \sigma'(y)$ for all *other* variable $y \in \text{Var} \setminus \{x\}$. For example, $[x \mapsto 2, y \mapsto 3] \sim_x [x \mapsto -100, y \mapsto 3]$ but $[x \mapsto 2, y \mapsto 3] \not\sim_x [x \mapsto 2, y \mapsto 4]$.

- *** 4. Let us suppose we introduce a new language construct for Boolean expressions so that the extended grammar is given as follows:

$$B \rightarrow \text{true} \mid \text{false} \mid B \ \&\& \ B \mid B \ \parallel \ B \mid !B \mid \text{forall } x. B$$

The denotational semantics for the new construct is given by the following equation:

$$\llbracket \text{forall } x. e \rrbracket_{\mathcal{B}}(\sigma) = \begin{cases} \top & \forall \sigma' \in \text{State}. \sigma \sim_x \sigma' \Rightarrow \llbracket e \rrbracket_{\mathcal{B}}(\sigma') \\ \perp & \text{otherwise} \end{cases}$$

with all other constructs retaining their original semantics.

- Evaluate the denotation of the expressions $\text{forall } x. x = 2$ and $\text{forall } x. x \leq y \parallel y \leq x$ in the state $[x \mapsto 2, y \mapsto 2]$.
- Prove that if $e \in \mathcal{B}$ and $e' \in \mathcal{B}$ are semantically equivalent Boolean expressions, then $\text{forall } x. e$ and $\text{forall } x. e'$ are semantically equivalent.
- Suppose that $e \in \mathcal{B}$ is a Boolean expression such that $\llbracket e \rrbracket_{\mathcal{B}}(\sigma) = \llbracket e \rrbracket_{\mathcal{B}}(\sigma')$ for any two states $\sigma \sim_x \sigma'$. Prove that $\text{forall } x. e$ is semantically equivalent to e .

Solution

- $\llbracket \text{forall } x. x = 2 \rrbracket_{\mathcal{B}}([x \mapsto 2, y \mapsto 2]) = \perp$ as $\llbracket x = 2 \rrbracket_{\mathcal{B}}([x \mapsto 3, y \mapsto 2]) = \text{false}$.
 - $\llbracket \text{forall } x. x \leq y \parallel y \leq x \rrbracket_{\mathcal{B}}([x \mapsto 2, y \mapsto 2]) = \top$ as, for any $n \in \mathbb{Z}$, either $n \leq 2$ or else $2 \leq n$.

- (b) Suppose $e \in \mathcal{B}$ and $e' \in \mathcal{B}$ are semantically equivalent Boolean expressions. That is $\llbracket e \rrbracket_{\mathcal{B}}(\sigma) = \llbracket e' \rrbracket_{\mathcal{B}}(\sigma)$ for all $\sigma \in \text{State}$. Let $\sigma \in \text{State}$ be some state. By definition $\llbracket \text{forall } x. e \rrbracket_{\mathcal{B}}(\sigma)$ is true if $\llbracket e \rrbracket_{\mathcal{B}}(\sigma') = \top$ for all $\sigma' \sim_x \sigma$. If this is the case, then $\llbracket e' \rrbracket_{\mathcal{B}}(\sigma') = \top$ for all $\sigma' \sim_x \sigma$ by semantic equivalence and thus $\llbracket \text{forall } x. e' \rrbracket_{\mathcal{B}}(\sigma)$ is also true. Conversely, if $\llbracket \text{forall } x. e \rrbracket_{\mathcal{B}}(\sigma)$ is false then $\llbracket e \rrbracket_{\mathcal{B}}(\sigma') = \perp$ for some $\sigma' \sim_x \sigma$. Therefore, $\llbracket e' \rrbracket_{\mathcal{B}}(\sigma') = \perp$ for some $\sigma' \sim_x \sigma$ and $\llbracket \text{forall } x. e' \rrbracket_{\mathcal{B}}(\sigma)$ is false as required. Therefore, $\text{forall } x. e$ and $\text{forall } x. e'$ are semantically equivalent.
- (c) Suppose that $e \in \mathcal{B}$ is a Boolean expression such that $\llbracket e \rrbracket_{\mathcal{B}}(\sigma) = \llbracket e \rrbracket_{\mathcal{B}}(\sigma')$ for any two states $\sigma \sim_x \sigma'$. Let $\sigma \in \text{State}$ be some state. If $\llbracket \text{forall } x. e \rrbracket_{\mathcal{B}}(\sigma)$ is true, then $\llbracket e \rrbracket_{\mathcal{B}}(\sigma') = \top$ for all $\sigma' \sim_x \sigma$. Therefore, $\llbracket e \rrbracket_{\mathcal{B}}(\sigma) = \top$ by assumption. Conversely, if $\llbracket \text{forall } x. e \rrbracket_{\mathcal{B}}(\sigma)$ is false, then $\llbracket e \rrbracket_{\mathcal{B}}(\sigma') = \perp$ for some $\sigma' \sim_x \sigma$. Therefore, $\llbracket e \rrbracket_{\mathcal{B}}(\sigma) = \perp$ by assumption. Thus, e and $\forall x. e$ are semantically equivalent as required.

*** 5. Consider the following program P :

while $1 \leq x$ do $\{x \leftarrow x - 1; y \leftarrow y + x\}$

By finding a suitable loop invariant, show that $\{x = n \ \&\& \ y = m\} P \{2y = 2m + n * (n - 1)\}$ for an arbitrary integers $n, m \geq 0$. Using this result, find two distinct states $\sigma, \sigma' \in \text{State}$ such that $\langle P, \sigma \rangle \rightarrow^* [y \mapsto 10]$ and likewise for $\langle P, \sigma' \rangle \rightarrow^* [y \mapsto 10]$.

Solution

The loop invariant is $I := x \geq 0 \ \&\& \ 2y = 2m + n(n - 1) - x(x - 1)$. To verify this, consider the strongest post-condition $\text{SPC}(y \leftarrow y + x; x \leftarrow x - 1, I \ \&\& \ 1 \leq x)$ which is equivalent to $2(y - x) = 2m + n(n - 1) - x(x + 1) \ \&\& \ 1 \leq x + 1$ and thus implies $2y = 2m + n(n - 1) - x(x - 1) \ \&\& \ x \geq 0$.

Using this loop invariant, we have that $\{x = n \ \&\& \ y = m\} P \{2y = 2m + n * (n - 1)\}$. As $n \geq 0$ and $2m = 2m + n(n - 1) - n(n - 1)$, and $2y = 2m + n(n - 1) - x(x - 1) \ \&\& \ x \geq 0 \ \&\& \ 1 > x$ implies $2y = 2m + n(n - 1)$.

To find two distinct states that leads to $[y \mapsto 10]$, we need to find $n, m \geq 0$ such that $20 = 2m + n(n - 1)$. The solutions are therefore:

- $[y \mapsto 10, x \mapsto 0]$
- $[y \mapsto 10, x \mapsto 1]$
- $[y \mapsto 9, x \mapsto 2]$
- $[y \mapsto 7, x \mapsto 3]$
- $[y \mapsto 4, x \mapsto 4]$
- $[y \mapsto 0, x \mapsto 5]$